

# DESIGN AND SIMULATION OF 4-BIT PROCESSOR

As a Part of the Industrial Project work submitted to Code Technologies for the

Internship

By

Jinkala Dhatri

from

Malla Reddy Engineering College (MREC)

## Abstract

The design of processors forms the foundation of modern digital systems and embedded applications. This project presents the design and simulation of a 4-bit processor using VHDL. The proposed processor consists of essential components such as an Arithmetic Logic Unit (ALU), register file, control unit, and memory, which work together to execute basic arithmetic and logical operations. The processor is designed to perform operations including addition, subtraction, AND, OR, and XOR based on a simple instruction set. VHDL is used to model the processor architecture, and the design is verified through behavioral simulation using Xilinx Vivado. Simulation results demonstrate the correct execution of instructions and validate the functionality of individual modules. The proposed 4-bit processor provides a simple and efficient platform for understanding processor architecture and digital system design. Furthermore, it serves as a foundation for developing more advanced processors for embedded and real-time applications.

**Keywords—** 4-Bit Processor, VHDL, FPGA, Arithmetic Logic Unit (ALU), Register File, Control Unit, Instruction Set Architecture, Xilinx Vivado, Digital System Design, Simulation..

## Contents

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>ABSTRACT</b>                                    | <b>ii</b> |
| <b>1. INTRODUCTION</b>                             | <b>3</b>  |
| 1.1 General                                        | 3         |
| 1.2 Objective                                      | 3         |
| <b>2. LITERATURE SURVEY</b>                        | <b>6</b>  |
| 2.1 Processor Architecture Fundamentals            | 6         |
| 2.2 ALU Design                                     | 6         |
| 2.3 Register File Organisation                     | 6         |
| 2.4 Control unit and Instruction Set Architecture  | 7         |
| 2.5 VHDL-Based Processor Design                    | 7         |
| 2.6 FPGA Implementation                            | 7         |
| <b>3. DESIGN AND SIMULATION OF 4-BIT PROCESSOR</b> | <b>8</b>  |
| 3.1 Hardware components                            | 8         |
| 3.1.1 FPGA Development board                       | 8         |
| 3.1.2 Light Emitting Diodes(LEDs)                  | 8         |
| 3.1.3 Push Button Switch                           | 9         |
| 3.1.4 Clock Source                                 | 9         |
| 3.1.5 USB Programming Cable                        | 9         |
| 3.1.6 Personal Computer                            | 9         |
| 3.2 Software Requirement                           | 10        |
| 3.2.1 Xilinx Vivado Design Suite                   | 10        |

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| 3.2.2 VHDL(VHSIC Hardware Description Language)                         | 10        |
| 3.3.3 Vivado simulator(XSim)                                            | 10        |
| 3.3 Block Diagram                                                       | 10        |
| 3.4 VHDL code                                                           | 11        |
| 3.5 TestBench Code                                                      | 14        |
| <b>4 IMPLEMENTATION OF DESIGN AND SIMULATION OF 4-BIT<br/>PROCESSOR</b> | <b>16</b> |
| 4.1 Working Mechanism                                                   | 16        |
| 4.2 Simulation Results                                                  | 16        |
| <b>5.Conclusion and Future scope</b>                                    | <b>17</b> |
| 5.1 Conclusion                                                          | 17        |
| 5.2 Future Scope                                                        | 17        |

**List of Figures**

|                      |    |
|----------------------|----|
| 1.FPGA board         | 8  |
| 2.LEDs               | 8  |
| 3.Push Button Switch | 9  |
| 4.Block Diagram      | 10 |
| 5.Simulation Result  | 16 |

# Chapter 1

## Introduction

### 1.1.General

A central processing unit (CPU) consists of an arithmetic logic unit (ALU), register file, control unit, and memory interfaces, as in the von Neumann architecture. The ALU performs arithmetic and logical operations on integer operands, and produces result values along with status flags (e.g. zero or carry-out). A small set of registers (the register file) holds temporary data and operands. The control unit decodes instructions and generates the control signals that orchestrate data flow through the CPU. In this project, we build a 4-bit accumulator-based processor: all computations use a single “accumulator” register (A) and data from a small memory. This simple design is suitable for demonstration and instruction, as early 4-bit CPUs were used in calculators and embedded controllers. Though 4-bit architectures are largely obsolete in modern systems, they serve as educational models

### 1.2.Objectives

The objective of this project is to design, implement, and simulate a simple 4-bit processor using VHDL on an FPGA platform. Specific goals include:

- Design the datapath components: a 4-bit ALU, accumulator register (A), program counter (PC), instruction ROM, and data RAM.
- Design the control logic to decode instructions and generate control signals.
- Implement each module in VHDL and integrate them into a top-level CPU entity.
- Simulate the processor in Xilinx Vivado to verify correct operation (instruction sequencing, ALU results, flag behaviour).
- Demonstrate operation with sample instructions and analyse the simulation waveforms.

## Chapter-2

### Literature Survey

#### 2.1. Processor Architecture Fundamentals

A CPU's primary functional units are the Arithmetic Logic Unit (ALU), a set of general-purpose registers (the register file), and the control unit. The ALU takes operand values and an opcode input, performs the specified operation, and outputs the result along with status flags (e.g. Zero, Carry). Typical ALU operations include addition, subtraction, logical AND/OR, and bit shifts. Registers store intermediate values: a typical simple CPU might have just a handful of registers, whereas more complex CPUs have many registers. The register file is a small memory of registers with multiplexers to route register values to the ALU inputs.

#### 2.2. ALU Design

The ALU is the core arithmetic-logic block. In a 4-bit ALU design, common approach is to use a 4-bit ripple-carry adder for addition/subtraction and parallel logic gates for bitwise operations. A multiplexer selects the desired output. For instance, one can implement four parallel logic circuits (e.g. bitwise AND, OR, XOR, NOT) on the operands and a separate adder circuit, then use a multiplexer to choose the result based on the opcode. Overflow (carry-out) is captured from the MSB of the adder, and a zero flag can be generated with a NOR of all result bits. The wired example ALU had an 8-to-1 multiplexer selecting among ADD, SUB, AND, OR, etc.

#### 2.3. Register File Organization

Small CPUs use a few registers. A typical simple design may have one accumulator register (A) and possibly one or two general registers or index registers. The register file circuit has input lines for write data and control, and multiplexers to select registers for reading. In our accumulator-based design, we effectively use a register file with one main register (A). However, this can be implemented as a one-word register file for modularity.

The register file (or accumulator) is clocked, updating its contents on write-enable signals. In larger CPUs, register files often have two read ports and one write port, but in a 4-bit CPU we typically read from A and one memory operand and write back to A.

### **2.4. Control Unit and Instruction Set Architecture (ISA)**

The Control Unit (CU) decodes the bits of each instruction to generate control signals for the datapath. For example, an instruction may specify “ADD value from memory into A”, so CU must: (1) enable memory read, (2) set ALU to ADD mode, (3) load the ALU output into register A, and (4) update flags. Conditional branch instructions (e.g. jump if zero) cause the CU to test ALU flags and update the PC accordingly.

### **2.5. VHDL-Based Processor Design**

VHDL is a hardware description language used to model digital logic at various abstraction levels. In VHDL, one can write concurrent code to describe combinational circuits, sequential processes for registers, and instantiate modules hierarchically. Using VHDL, designers simulate the behavior of circuits before synthesizing into FPGA logic. It supports dataflow, behavioral, and structural styles. Many academic projects use VHDL to teach CPU design (e.g. designing program counters, ALUs, and simple processors in VHDL). VHDL’s concurrency and strong typing help catch design errors early. In our work, each component (ALU, register, memory, control) is coded as a VHDL module, then connected in a top-level entity.

### **2.6. FPGA Implementation**

FPGAs provide a reconfigurable hardware fabric of logic blocks and memory, well-suited for implementing custom CPUs. The design flow uses tools (like Xilinx Vivado) to synthesize VHDL code into FPGA logic elements, simulate waveforms, and view RTL schematics. Vivado supports timing analysis and hardware verification. FPGAs allow full parallelism: all combinational logic (ALU, multiplexers) operates simultaneously each clock. Our 4-bit processor can be fully implemented on a small FPGA or CPLD. In practice, dedicated on-chip features (e.g. block RAM) could implement the instruction and data memory.

## Chapter-3

# DESIGN AND SIMULATION OF 4-BIT PROCESSOR

### 3.1. Hardware Components

#### 3.1.1. FPGA Development Board

A typical FPGA board (e.g. a Xilinx Spartan or Artix series) is used to implement the design. It provides the logic fabric, clock source, I/O pins, and a USB programming interface. The exact model is not crucial for simulation but would be used for hardware testing.



Fig1.FPGA board

#### 3.1.2. Light Emitting Diodes (LEDs)

On real hardware, a set of LEDs or seven-segment displays can show the processor outputs. For instance, four LEDs might display the accumulator's current 4-bit value or a status code. These are not strictly needed for simulation but would help visualize results on a board

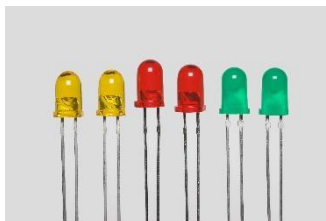


Fig2.LEDs



### 3.1.3. Push Button Switch

A push button switch is used to generate the emergency vehicle signal. When the button is pressed, the controller activates the emergency mode and immediately changes the traffic light to Green, providing priority access to emergency vehicles.



Fig3.Push Button Switch

### 3.1.4. Clock Source

The FPGA provides a system clock (e.g. 50 MHz). We use a clock input to drive the sequential circuits (PC, registers, etc.) in the CPU.

### 3.1.5. USB Programming Cable

A USB cable is used to connect the FPGA board to a computer. It enables downloading the VHDL design onto the FPGA and facilitates communication between the hardware and development software.

### 3.1.6. Personal Computer (PC/Laptop)

A computer is required for writing VHDL code, simulating the design, synthesizing the circuit, and programming the FPGA board. It serves as the development platform for the entire project.

## 3.2. Software requirement:

### 3.2.1. Xilinx Vivado Design Suite

VHDL coding, synthesis, and simulation environment for FPGA development. We use Vivado to write VHDL, compile modules, simulate waveforms, and analyze timing/RTL.

### 3.2.2. VHDL (VHSIC Hardware Description Language)

Hardware Description Language used to implement the CPU's logic and simulate its behavior at the register-transfer level.

### 3.2.3. Vivado Simulator (XSim)

Built-in RTL simulator in Vivado, used to verify the functionality of the VHDL design before hardware deployment.

### 3.3. Block Diagram:

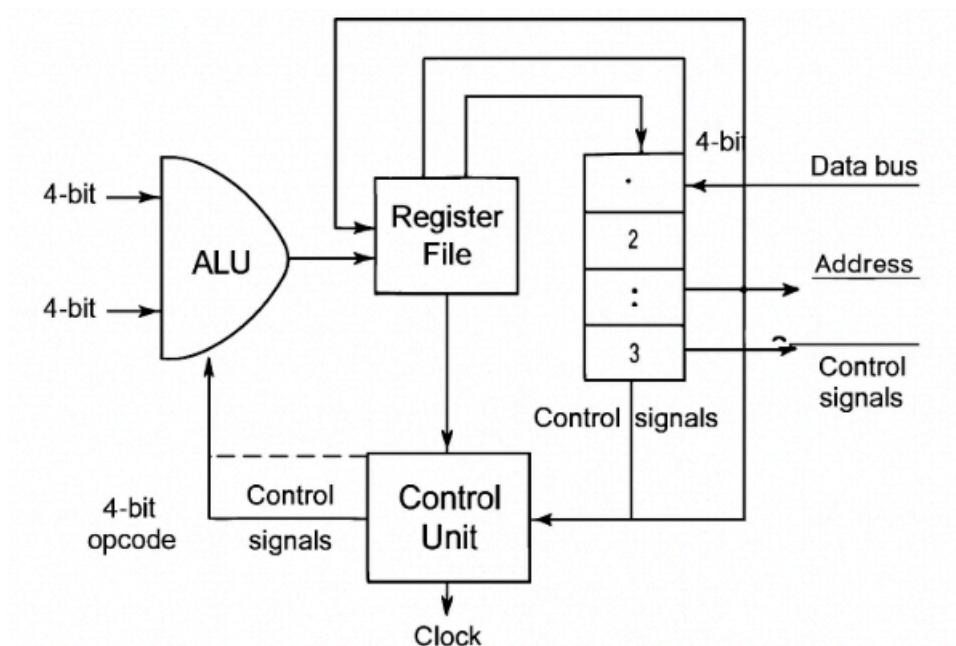


Fig4. Block Diagram

### 3.4. VHDL code:

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity alu is
    Port(
```

```
A    : in STD_LOGIC_VECTOR(3 downto 0);
B    : in STD_LOGIC_VECTOR(3 downto 0);
Opcode : in STD_LOGIC_VECTOR(3 downto 0);
Result : out STD_LOGIC_VECTOR(3 downto 0)
);
end alu;

architecture Behavioral of alu is
begin
process(A,B,Opcode)
begin
case Opcode is
when "0000" =>
    Result <= std_logic_vector(unsigned(A)+unsigned(B));
when "0001" =>
    Result <= std_logic_vector(unsigned(A)-unsigned(B));
when "0010" =>
    Result <= A and B;
when "0011" =>
    Result <= A or B;
when "0100" =>
    Result <= A xor B;
when others =>
```

```

    Result <= "0000";

end case;

end process;

end Behavioral;

```

### 3.4.Testbench code:

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity alu_tb is

end alu_tb;

architecture Behavioral of alu_tb is

    signal A      : std_logic_vector(3 downto 0);

    signal B      : std_logic_vector(3 downto 0);

    signal Opcode : std_logic_vector(3 downto 0);

    signal Result : std_logic_vector(3 downto 0);

begin

    uut : entity work.alu

    port map(

        A => A,

        B => B,

        Opcode => Opcode,

        Result => Result

```

```
);  
  
process  
  
begin  
  
A <= "0101";  
  
B <= "0011";  
  
Opcode <= "0000";  
  
wait for 50 ns;  
  
Opcode <= "0001";  
  
wait for 50 ns;  
  
Opcode <= "0010";  
  
wait for 50 ns;  
  
Opcode <= "0011";  
  
wait for 50 ns;  
  
Opcode <= "0100";  
  
wait for 50 ns;  
  
wait;  
  
end process;  
  
end Behavioral;
```

## Chapter-4

# IMPLEMENTATION OF DESIGN AND SIMULATION OF 4-BIT PROCESSOR

### 4.1.Working Mechanism:

On reset, the Program Counter (PC) is set to 0, and the instruction ROM outputs the instruction at address 0. Each clock cycle, the current IR is executed. For example, if IR holds an ADD instruction, the ALU adds the accumulator value (A) and the memory operand, updates the Zero/Carry flags, and stores the sum back into A. Meanwhile, the control logic advances the PC by 1 or sets it to a target address for jump instructions. Conditional jumps inspect the Zero flag: a JZ instruction will cause the PC to jump if Zero=1; otherwise PC just increments. The register file (RegFile) loads the ALU result into A only when the AccWrite signal is high. Memory writes (STA) occur when MemWrite is asserted, writing the accumulator's value into RAM. Thus in each cycle, the CPU fetches, decodes, executes, and writes back. Over many cycles it follows the program flow stored in ROM.

### 4.2.Simulation Results:

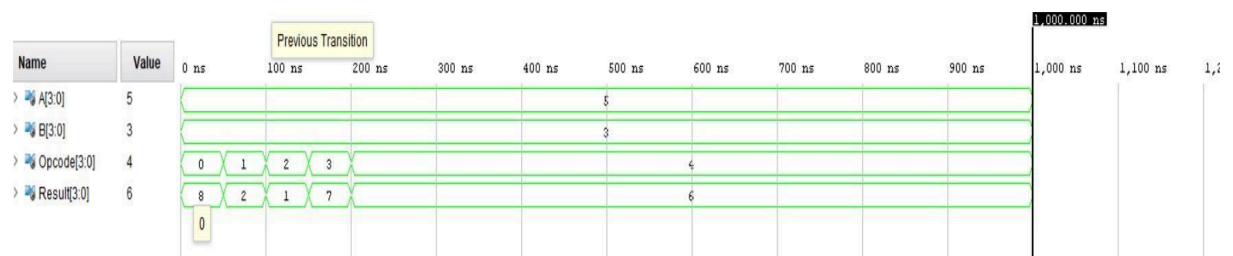


Fig5.Simulation Result

## Chapter-5

### Conclusion And Future scope

#### 5.1.Conclusion:

We have successfully designed and simulated a simple 4-bit processor using VHDL on an FPGA. The design implements an accumulator architecture with a basic instruction set (load, store, add, sub, and, or, jumps) and a finite-state control mechanism. Simulation in Xilinx Vivado confirmed correct operation: instructions were fetched and executed properly, ALU results were correct, and flags governed branches as expected. This project demonstrates how FPGA technology and VHDL can realize a complete CPU, even with minimal resources. It provides a hands-on example of processor architecture for educational purposes, reinforcing concepts of datapath design and control logic.

#### 5.2.Future Scope:

This 4-bit CPU can be extended in several ways. A richer instruction set could be added (e.g. multiplication, I/O operations, subroutine call/return). We could expand from accumulator to multiple registers or implement a small stack. Pipelining or multi-cycle control could improve performance or illustrate advanced concepts. Interfaces to on-board I/O (buttons, displays, serial UART) would make it interactive on FPGA hardware. One could also design assemblers or compilers targeting this ISA for higher-level programming.